Minor Project – III (MCA-269): Software Requirements Specification

Project	AI Tutor: An Evidence-Based Adaptive Learning System with Deliberate Scaffolding Withdrawal
Programme	Master of Computer Applications, Semester III, Batch 2025–2027
Institution	Jagan Institute of Management Studies, Rohini
Submitted by	Ashwani Kumar Puri (Roll No. _____)
Guide	_____
Date	24 September 2026 (first draft 7 September 2026)
Submission	Phase 1: Synopsis, Problem Analysis, Literature Survey, SRS

This SRS follows the IEEE 830 layout, adjusted to match the module list in the synopsis. DFD, ER and UML diagrams are part of the Phase 2 submission and are not repeated here. This document says what the system must do, not how its data or components are organised inside.

1. Introduction

1.1 Purpose

This document lists the functional and non-functional requirements for version 1 of AI Tutor. Design, implementation and testing in Phases 2 and 3 will be checked against these numbered requirements. It takes the objectives O1 to O7 from the Problem Analysis (§6) and the module list from the Synopsis (§4) and turns them into requirements.

1.2 Conventions

- Functional requirements are numbered FR-(module)-(number), for example FR-M4-03.
- Each requirement has a MoSCoW priority: Must, Should or Could. We have not listed any "Won't" items here; those are in the out-of-scope list in Problem Analysis §7.2.
- "Shall" means the requirement is mandatory. "Should" means it is recommended.

1.3 Who this is for

The project team, as a reference for design and implementation, and the guide and evaluation panel for the Phase 1 review. The department guidelines ask that documentation follow professional software practice, so we have tried to write this the way a working team would write its SRS.

1.4 Scope

As set in Problem Analysis §7: one subject (Data Structures and Algorithms, with topic coverage checked against the roadmap.sh DSA roadmap and prerequisites written by us), text, code and

exercises only, the full learner model and hint engine, sandboxed code execution, and basic editing and admin screens. The out-of-scope list in Problem Analysis §7.2 applies as it is.

1.5 Related documents

- Synopsis: system description, modules, tools and methodology.
- Literature Survey: the research behind each design decision.
- Problem Analysis: problem definition, root causes, objectives O1 to O7, feasibility and risks.

2. Overall Description

2.1 Product perspective

This is a new, standalone system. It does not replace an older application; Problem Analysis §3 describes a manual process spread across many tools. It relies on two outside services that we do not build: a language model API (a small model for hints and grading, a larger one for generating exercises) and a managed code-execution sandbox.

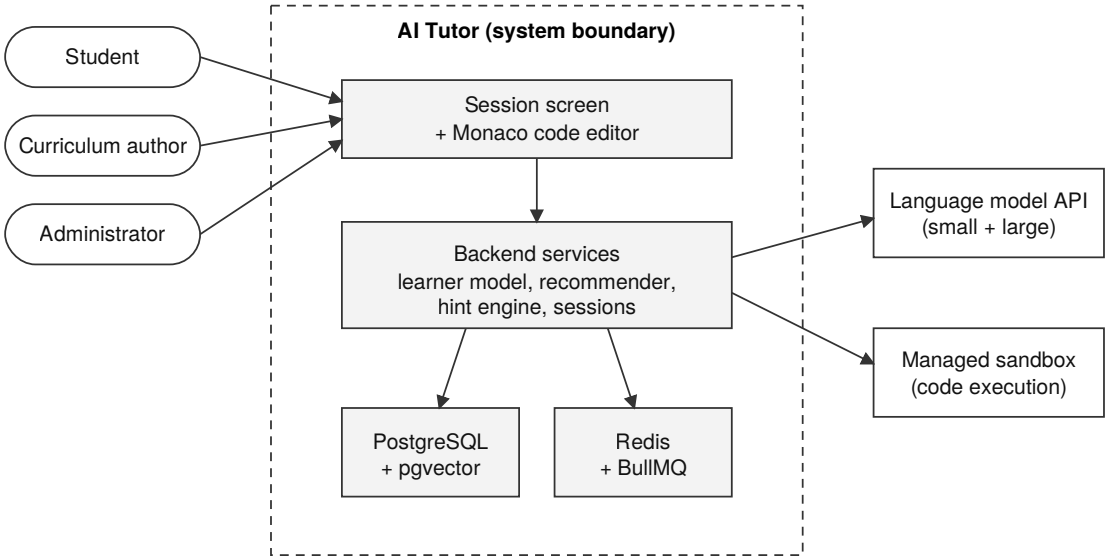


Figure 1: System context

2.2 Main functions

In brief, the system:

- tests a new student to find their starting level,
- keeps an evidence-based mastery score per skill that fades over time,
- picks the next step from the prerequisite graph,
- generates an exercise at the right level and checks it before use,
- runs and grades submitted code in a sandbox,
- gives step-by-step hints with limits enforced on the server,
- schedules revision before a skill is forgotten,

- shows honest progress,
- lets an author maintain the skill graph, and
- lets an admin watch prediction accuracy and cost.

Section 3 gives the full list.

2.3 Users

User	Description	How often
Student	Learns DSA on their own; no prior experience with tutoring systems assumed	Main user, daily sessions
Curriculum author	Comfortable with technical material; writes and maintains the skill graph and exercise templates	From time to time
Administrator	Runs the platform; watches accuracy, cost and generation failures	From time to time

Version 1 fully supports the student. The project team takes the author and admin roles (synopsis §3.7).

2.4 Operating environment

Runs in a web browser with nothing to install (synopsis §3). The backend runs on a cloud-hosted managed platform with managed PostgreSQL and a managed sandbox service. Development machine: multi-core x86-64 with 8 or more threads, at least 16 GB RAM, 512 GB SSD (synopsis §5.1).

2.5 Design constraints

- The next step must be chosen by the system's own rules and never handed to a language model (synopsis §3.4). This keeps the recommender explainable and easy to audit.
- Mastery must never change because content was read, watched or marked complete (Problem Analysis, O1).
- The current allowed hint level must be stored on the server. It must not depend on the client or on the prompt (Problem Analysis, O3).
- We will not build our own sandbox in version 1 (Problem Analysis §7.2). Using a managed service is a firm constraint.

2.6 Assumptions and dependencies

- The language model API and the sandbox service are available and fast enough for interactive use. Both are outside our control.
- At least one student other than the author will be available to complete the validation check (synopsis §7.5, risk R5) before the final submission.
- The team and its division of work (synopsis §4.1) stay the same through implementation.

3. Functional Requirements

Grouped by the modules listed in synopsis §4.

M1: Login and Accounts (Must)

Sign-up, login, sessions and the student's goal.

- **FR-M1-01** (Must): The system shall let a student register with a unique ID and a password.
- **FR-M1-02** (Must): The system shall log in and keep a session for each type of user (student, author, admin).
- **FR-M1-03** (Must): The system shall record the student's goal during profile setup, for the recommender (M5) to use.
- **FR-M1-04** (Must): The system shall use role-based access so that students cannot reach author or admin screens.
- **FR-M1-05** (Should): The system should let users reset their password without help from an admin.

M2: Skill Graph (Must)

Stores the prerequisite graph and provides an editing screen.

- **FR-M2-01** (Must): The system shall store skills as nodes of a directed acyclic graph with prerequisite edges.
- **FR-M2-02** (Must): The system shall reject any edit that would create a cycle.
- **FR-M2-03** (Must): The system shall provide a screen to add, edit and retire skills and edges.
- **FR-M2-04** (Must): The system shall mark each skill as syntax-type or concept-type, which M4 uses for the decay rate.
- **FR-M2-05** (Could): The system could keep a history of changes to the graph.

M3: Diagnostic Test (Must)

Places a new student in the graph.

- **FR-M3-01** (Must): The system shall run an adaptive test that estimates starting reading and writing scores for each skill it covers.
- **FR-M3-02** (Must): The system shall use the test results as the starting values in the mastery model (M4).
- **FR-M3-03** (Should): The system should let a student take the test again.

M4: Mastery Model (Must)

The central data model. Every other module reads from it or writes to it.

- **FR-M4-01** (Must): The system shall store, for each student and skill, a reading score, a writing score, the time of the last evidence, attempt and success counts, and a decay rate.

- **FR-M4-02** (Must): The system shall change a mastery score only on evidence from execution (a passed test run, a correct prediction of program behaviour, or a solved exercise), and never because content was read, watched or marked complete.
- **FR-M4-03** (Must): The system shall calculate current mastery as the stored score reduced exponentially by the time since the last evidence, using the skill's decay rate.
- **FR-M4-04** (Must): The system shall add a skill to the revision queue (M10) when its current mastery falls below a set threshold.
- **FR-M4-05** (Must): The system shall weight new evidence by the exercise level, the number of hints used and the time taken, before updating the stored score.

M5: Next-Step Recommender (Must)

Chooses the student's next step using fixed rules.

- **FR-M5-01** (Must): The system shall find candidate skills whose prerequisites meet a mastery threshold.
- **FR-M5-02** (Must): The system shall include skills due for revision (from M10) as candidates, along with new skills.
- **FR-M5-03** (Must): The system shall score each candidate on prerequisite readiness, relevance to the student's goal, urgency of revision and how recently it was covered.
- **FR-M5-04** (Must): The system shall pick the highest-scoring candidate itself, without asking a language model to choose.
- **FR-M5-05** (Must): The system shall set the exercise level (1 to 5) from the student's current writing score for the chosen skill.

M6: Exercise Generation and Checking (Must)

Generates exercises for each student and level, and checks each one before use.

- **FR-M6-01** (Must): The system shall generate exercise content for the chosen skill and level using the larger language model.
- **FR-M6-02** (Must): The system shall run the reference solution against the generated tests before showing any exercise to a student, and shall discard exercises that fail.
- **FR-M6-03** (Must): The system shall use a hand-written exercise from the template bank when generation or checking fails.
- **FR-M6-04** (Should): The system should log every failed generation for the analytics reports (M13).

M7: Sandbox and Grading (Must)

Runs student code in isolation.

- **FR-M7-01** (Must): The system shall run submitted code only inside an isolated sandbox with no network access and limited resources.

- **FR-M7-02** (Must): The system shall run submitted code against the exercise's tests and give a pass or fail result for each test.
- **FR-M7-03** (Must): The system shall apply a time limit and a memory limit to each exercise, and shall report "limit exceeded" separately from "wrong answer". This way, a correct but slow solution is recorded as different evidence from an efficient one.
- **FR-M7-04** (Must): The system shall send a structured result to the mastery model (M4) and the session handler (M9).

M8: Hint Engine (Must)

Makes sure solutions are not handed out, using the system's design and not only the prompt.

- **FR-M8-01** (Must): The system shall store the current allowed hint level on the server, for each student and each attempt.
- **FR-M8-02** (Must): The system shall send the student only the content for the current hint level, never the whole ladder.
- **FR-M8-03** (Must): The system shall raise the hint level only when the student asks for it, never on its own.
- **FR-M8-04** (Must): The system shall check every generated reply for code that looks like the solution to the current exercise, and block the reply if it does.
- **FR-M8-05** (Must): The system shall log each time the hint ladder reaches the last level (the answer) and pass this to the analytics module (M13).

M9: Session Handling (Must)

Session lifecycle and order of steps.

- **FR-M9-01** (Must): The system shall let a student start, pause, resume and end a session.
- **FR-M9-02** (Must): The system shall order the steps in a session according to the recommender (M5).
- **FR-M9-03** (Must): The system shall save progress so that an interrupted session continues from the same step.

M10: Revision Scheduler (Must)

Schedules revision from the decay model.

- **FR-M10-01** (Must): The system shall work out each skill's revision date from its decay rate and the time of the last evidence (M4).
- **FR-M10-02** (Must): The system shall make the revision queue available to the recommender (M5).

M11: Progress View (Must)

Shows the student's real level without inflating it.

- **FR-M11-01** (Must): The system shall show the skill graph with each node coloured by current mastery.
- **FR-M11-02** (Must): The system shall show reading and writing scores separately for each skill, never combined.
- **FR-M11-03** (Must): The system shall show weak areas as they are. The value displayed shall be the stored current mastery, with no adjustment.

M12: Support Fade Controller (Must)

Reduces help on a planned schedule.

- **FR-M12-01** (Must): The system shall store a configurable schedule for reducing support as part of the student's plan.
- **FR-M12-02** (Must): The system shall apply, in every session, the maximum help level and exercise level allowed at that point in the schedule, even if the student asks for more.

M13: Accuracy and Analytics (Should)

Lets the admin check how accurate the system's own predictions are.

- **FR-M13-01** (Should): The system should compare predicted mastery with results on held-back test exercises.
- **FR-M13-02** (Should): The system should show the admin trends in hint use.
- **FR-M13-03** (Should): The system should report exercise generation success and failure rates by skill and level.

4. External Interfaces

4.1 User interfaces

- For the student: a responsive web session screen with the Monaco editor (the editor used in VS Code).
- For the author: a screen for editing the graph.
- For the admin: dashboards for accuracy, cost and generation failures.

There is no mobile app in version 1 (Problem Analysis §7.2).

4.2 Hardware interfaces

None beyond a normal computer with a modern browser. The development machine is described in synopsis §5.1.

4.3 Software interfaces

Interface	Direction	Purpose
Language model API, small model	Backend to external	Hints and help with grading (many calls)
Language model API, large model	Backend to external	Generating exercises and explanations (few calls, cost-limited)
Managed sandbox service	Backend to external	Running and grading submitted code in isolation
PostgreSQL with pgvector	Backend, internal	Tables for the skill graph and mastery records; vector search for topic content
Redis with BullMQ	Backend, internal	Session state, revision queue, background jobs for generation and code runs

4.4 Communication

The frontend and backend talk over HTTPS using REST and JSON. Generation and sandbox jobs are passed internally through the Redis queue.

5. Non-Functional Requirements

5.1 Performance

- **NFR-P1:** Hint replies (M8) should come back within a few seconds, since they use the small model and the student is waiting on them. We will set an exact target in Phase 2 after measuring real response times.
- **NFR-P2:** Exercise generation and checking (M6) can take longer, because it runs in the background through the job queue and does not block the session screen.

5.2 Security

- **NFR-S1:** The sandbox (M7) shall have no network access and shall apply resource and time limits to every run.
- **NFR-S2:** The hint ladder and output filter (M8) shall be enforced on the server. A limit that exists only on the client or in the prompt does not count.
- **NFR-S3:** Passwords shall never be stored as plain text.
- **NFR-S4:** Role checks (M1) shall be applied on every author and admin API endpoint, not only in the user interface.

5.3 Reliability

- **NFR-R1:** If exercise generation fails, the system shall always use the template bank (M6) and not show an error to the student during a session.

- **NFR-R2:** The result returned from the sandbox shall make it clear whether the sandbox itself failed or the student's code failed a test.

5.4 Usability and accessibility

- **NFR-U1:** The code editor shall be usable with the keyboard alone. Monaco supports this already.
- **NFR-U2:** The progress view (M11) shall not rely on colour alone to show information.

5.5 Maintainability

- **NFR-M1:** The recommender and mastery engine shall not depend on any particular subject, so new subjects (synopsis §8, item 1) can be added without changing the engine.

5.6 Scalability

- **NFR-SC1:** Generation and sandbox requests shall go through the background job queue (BullMQ) and not be handled directly in the request, so that they can scale separately from interactive requests.

6. Data Requirements

These are the main data entities at a conceptual level. The detailed ER diagram is part of Phase 2.

Entity	What it stores
Learner	A registered student and their goal
SkillNode	One skill in the prerequisite graph, with its type (syntax or concept)
PrerequisiteEdge	A dependency from one skill to another
MasteryRecord	One student and one skill: reading score, writing score, time of last evidence, decay rate, attempts and successes
Exercise	One generated or template exercise, tagged with skill and level
ExerciseAttempt	One student's submission for one exercise, with its result
HintEvent	One step up the hint ladder in an attempt, including whether it reached the answer
Session	One study session: its state and current step
ReviewQueueEntry	A skill due for revision for a given student
DiagnosticResult	The result of the starting test
CalibrationRecord	Predicted and actual result on a held-back test exercise

7. Use Cases

ID	Use case	Main actor	Modules
UC-1	Register or log in	Student, author, admin	M1

ID	Use case	Main actor	Modules
UC-2	Take the diagnostic test	Student	M3
UC-3	Get the next recommended step	Student	M5, M6, M9
UC-4	Submit a solution for grading	Student	M7
UC-5	Ask for a hint	Student	M8
UC-6	View progress and the mastery map	Student	M11
UC-7	Create or edit the skill graph	Curriculum author	M2
UC-8	Set the support fade schedule	Admin or author	M12
UC-9	Look at accuracy reports	Admin	M13
UC-10	Resume an interrupted session	Student	M9

Example: UC-5, Ask for a hint

Actor: Student

Before: The student has an active exercise attempt (M9) and is stuck.

Main steps:

1. The student clicks "Ask for a hint" in the current exercise.
2. The hint engine (M8) reads the allowed hint level for this attempt.
3. The system checks the support fade schedule (M12) to make sure this level is allowed in the current session.
4. The system fetches or generates the content for that one level using the small model. The levels, in order, are: concept, where to look, technique, worked example, answer.
5. The system checks the reply for code that looks like the solution (FR-M8-04). If it finds any, it generates the reply again or uses the template hint for that level.
6. The system sends only that level's content to the student and raises the stored level for the next request.

Other path: If the requested level is the last one (the answer), the system logs it (FR-M8-05) before replying.

After: The hint count for the attempt is updated. M4 uses it to weight the evidence when the attempt ends (FR-M4-05).

8. Traceability Matrix

Problem (Problem Analysis §4)	Objective (Problem Analysis §6)	Module	Example requirement
D1: completion measured, not ability	O1	M4	FR-M4-02
D2: recognition trained, not recall	O1, O5	M4, M6	FR-M4-02, FR-M6-02

Problem (Problem Analysis §4)	Objective (Problem Analysis §6)	Module	Example requirement
D3: linear courses	O2	M5	FR-M5-04
D4: no learner model	O1, O2	M4, M5	FR-M4-01
D5: unrestricted AI help	O3, O6	M8, M12	FR-M8-01, FR-M12-02
D6: forgetting without revision	O4	M4, M10	FR-M4-03, FR-M10-01
D7: the empty file ends sessions	O5, O6	M5, M12	FR-M5-05, FR-M12-01

9. Appendix

9.1 Glossary

- **Current mastery:** the stored mastery score after decay over time. This is the value the recommender uses.
- **Level (rung):** one of the five exercise difficulty levels: Predict, Diagnose, Complete, Extend, Build.
- **Answer reveal:** the hint ladder reaching its last level (the answer) during one attempt.

9.2 Items for Phase 2

- Set exact performance targets for NFR-P1 and NFR-P2 after measuring real response times.
- Decide the exercise language for version 1 and record the decision.
- Prepare the DFD, ER and UML diagrams that this SRS leaves out.